- **Belief Update**: Based on the above decision-making process and feedback, the agent refines b_t as follows,

$$b_{t+1} = L_B(b_t, m_{t+1}, a_t, r_t, o_{t+1}). \tag{7}$$

The agent ultimately receives a cumulative discounted reward (i.e., $\sum_{t=0}^{\infty} \gamma^t r_t$) as a measure of goal achievement.

Different from the single-agent setting, in the LLM-based MAS, each agent must communicate with other agents to accomplish goals. To model these processes, a communication structure G_t is introduced among N agents. Furthermore, G_t would evolve with time to align with the dynamic adjustments of MAS. Therefore, compared with the above loop, the loop of LLM-based MAS is given as "Reasoning–Execution–**Broadcasting**–Feedback–Environment Transition–Perception–Memorization–Belief Update–**Structure Evolution**". This loop includes two additional procedures: Broadcasting and Structure Evolution. Broadcasting means that the agent broadcasts its message to neighboring nodes according to its plan, while structure evolution indicates that the communication structure G_t can be updated in each iteration. The complete description of the loop of MAS is given in Appendix A.

Definition of Agent hallucinations

According to the proposed internal–external distinction, agent hallucinations refer to fictitious or erroneous operations in LLM-based agents, driven by overconfident "human-like behaviors" in reasoning, execution, perception, memorization, and communication. Unlike LLM hallucinations, which are typically limited to linguistic errors, agent hallucinations go beyond language generation, manifesting as compound deviations arising from the coupling and interaction of multiple modules. These deviations can escalate to task failures and even physical safety risks. In contrast to other types of agent errors, agent hallucinations are often spontaneously generated within the agent's own cognitive processes, representing an endogenous error form. Furthermore, owing to their cross-module propagation and hidden cognitive nature, agent hallucinations pose an especially serious threat, making them a central challenge in the pursuit of agent safety.

III. Taxonomy of Agent Hallucinations

We first provide a formal definition of agent hallucinations. Based on this definition, we then introduce the new taxonomy that includes the following five types of agent hallucinations: Reasoning Hallucinations, Execution Hallucinations, Perception hallucinations, Memorization Hallucinations, and Communication Hallucinations, as illustrated in Fig. 2. Detailed descriptions of each type of agent hallucinations are provided in the following sub-sections. Representative hallucination examples are presented in Appendix B.

A. Reasoning Hallucinations

Reasoning serves as the cornerstone of an agent's functionality, influencing behavior analysis and decision-making [40], [41], [42]. Upon receiving a specific goal g, the agent first leverages its own reasoning capabilities to perform goal understanding for inferring the user's true intention:

- **Goal Understanding**: This phase occurs before the agent executes multiple loops:

$$\mathcal{I} = \text{Understand}(b_0, g), \tag{8}$$

where $\mathcal{I}$ denotes the inferred intention, and b_0 represents the initial belief state. When $\mathcal{I}$ is complex and relatively difficult to execute, the agent would perform the intention decomposition to decompose $\mathcal{I}$ into a series of manageable sub-intentions $\{\mathcal{I}_t\}_{t=0}^{n}$.

- **Intention Decomposition**: Typically, there are two decomposition methods: pre-defined decomposition and dynamic decomposition. In the first one, these sub-intentions are specified in advance:

$$\{\mathcal{I}_t\}_{t=0}^{n} = \text{Pre-decompose}(b_0, \mathcal{I}). \tag{9}$$

In the second one, the sub-intention for each loop is generated by

$$\mathcal{I}_t = \text{Dyn-decompose}(b_t, \mathcal{I}). \tag{10}$$

The decomposition results are dynamically optimized based on the current belief state b_t, enabling timely and continuous refinement for the subsequent workflow [43].

- **Planning Generation**: Each sub-intention $\mathcal{I}_t$ is then mapped to a concrete plan p_t for the current loop:

$$p_t = \text{Plan}(b_t, \mathcal{I}_t). \tag{11}$$

Definition of Reasoning Hallucinations

Reasoning hallucinations refer to the phenomenons where LLM-based agents generate seemingly plausible plans which are in fact logically flawed or unsupported. Reasoning hallucinations occur in either of above three stages, and we categorize them into three types: **Goal Understanding Hallucinations (GUHs)**, manifested as misinterpretation of goal semantics, typically caused by problematic objective expression and inadequate subjective comprehension; **Intention Decomposition Hallucinations (IDHs)**, characterized by the flawed decomposition processes that produce irrelevant or infeasible sub-intentions, often due to deficient dependency modeling; **Planning Generation Hallucinations (PGHs)** represent hallucinations that occur when an agent generates each specific plan of a sub-intention, typically caused by the misinterpretation and misapplication of planning information.

The subsequent sub-sections detail the triggering causes that give rise to these three types of reasoning hallucinations.

1) Problematic Objective Expression. Understanding the user's true intention from g is the first and essential step for effective

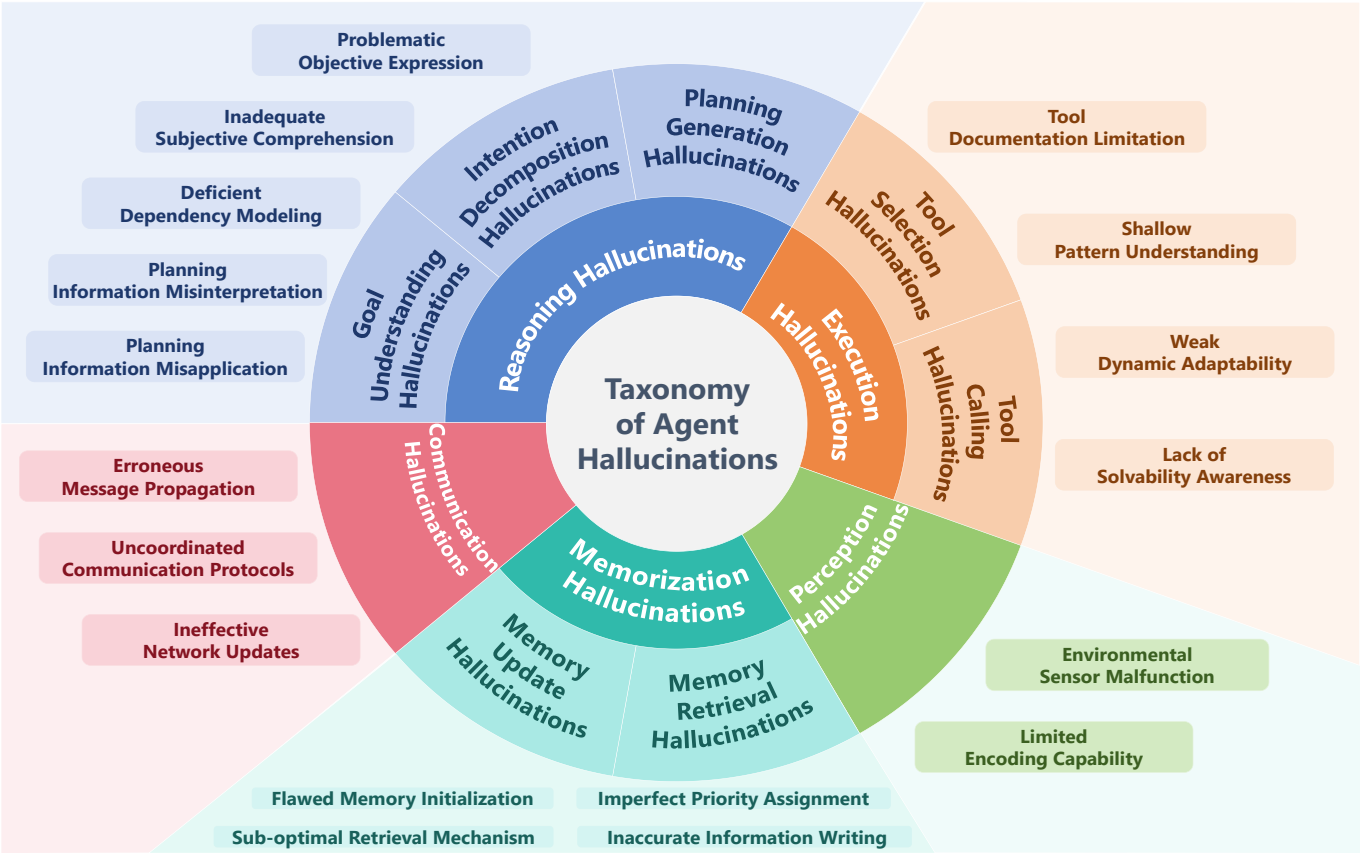

Fig. 2. A taxonomy of agent hallucinations. It includes five hallucination types and nine hallucination sub-types with corresponding triggering causes.

reasoning [44]. Objectively, when the expression of goal information carries a certain degree of semantic vagueness, it can easily lead to erroneous parsing of user intention and induce reasoning hallucinations [45], [46]. This semantic vagueness can be mainly attributed to two issues: **Incomplete Goal Specification** and **Ambiguous Content Presence** [47], [48]. To address semantic vagueness in user input, a plausible and effective approach is to endow the agent with the capability of active clarification [49], [50]. By engaging in appropriate proactive interactions with the user, the agent can gain a deeper understanding of the user's true needs and intentions.

2) Inadequate Subjective Comprehension. In addition to above objective issues, there also exist subjective limitations in the agent's comprehension capability. These are mainly manifested as **Instruction-following Deviation** [51], as well as **Long-range Contextual Misuse** [52]. Specifically, instruction following refers to an agent's ability to adhere to user-provided instructions in order to accurately comprehend intentions [53]. When this procedure breaks down, the agent struggles to segment critical fields and extract key information from instructions, leading to distorted understanding and reasoning hallucinations [54], [55], [56]. Additionally, when agents handle excessively long input goals, they may fail to capture long-range contexts and instead over-rely on the most recent tokens, resulting in contextual misuse [57], [58], [59]. Such deficiencies cause agents to violate explicitly stated user preferences [60].

3) Deficient Dependency Modeling. Intention decomposition is a crucial phase where agents determine the successive reasoning process via the decomposed sub-intentions [61], [62], [63]. These sub-intentions typically form a sequentially dependent chain, where the completion of each sub-intention pre-supposes the successful fulfillment of its predecessors. Inadequate modeling of dependency relationships among these sub-intentions can give rise to three types of errors: **Sub-intention Omission**, **Sub-intention Redundancy** [64], [65] and **Sub-intentions Disorder**. All three severely compromise reasoning integrity and efficiency, ultimately leading to reasoning hallucinations [66]. Sub-intention omission occurs when agents fail to identify essential sub-intentions, leading to the absence of critical reasoning steps [67]. Sub-intention redundancy arises when agents erroneously introduce task-irrelevant sub-intentions, causing reasoning processes to deviate from the core intention and generating hallucinatory content [68], [69], [67]. Sub-intention disorder refers to the phenomenon in which an agent arranges sub-intentions with a sequential relationship in the wrong order during reasoning, causing it to fail to obtain the results of prerequisite sub-intentions, thereby missing key information and further generating hallucinations [70].

4) Planning Information Misinterpretation. In Eq.(11), the agent relies on two key sources of information to generate sub-intention plans: **Operable Objects** and **Self-knowledge**. For operable objects, the agent must recognize both explicit and implicit relational connections between entities (e.g.,

spatial, temporal, or social relationships) [71], [72], [73]. The misinterpreting information related to these operable objects can lead the agent to construct plans based on incorrect assumptions, thereby inducing planning generation hallucinations [74]. With respect to self-knowledge, the agent is expected to plan reasonably within the bound of its own self-knowledge to avoid planning errors. When confronted with planning problems beyond its knowledge boundary, the agent tends to respond with excessive confidence, generating answers that sound certain but are actually incorrect [75].

5) Planning Information Misapplication. Even when an agent correctly identifies the operable objects within its knowledge boundary, its ability to effectively utilize them remains constrained by its **Logical Reasoning** and **Introspection Capacity** [76]. Logical fallacies primarily manifest as rule conflicts and causal misattributions [77]. Similar to humans, when agents encounter issues such as rule conflicts, causal inversion, or misattribution during causal reasoning [78], they often fail to complete the reasoning process or resolve underlying logical paradoxes, ultimately leading to erroneous planning [79]. Additionally, introspection is one of the key mechanisms by which agents achieve learning, cognition, and knowledge updating [15], agents that lack sufficient introspection capacity are unable to effectively improve their performance by revisiting logical fallacies in their reasoning [80]. Such agents may misinterpret feedback signals and accurately identify the sources of error [81], thereby making misguided adjustments to their reasoning processes [82]. Even when feedback is correctly interpreted, the introspection process may still be flawed: agents might wrongly judge their reasoning as consistent despite existing logical contradictions, or mistakenly believe they have considered all relevant factors while overlooking critical variables [83].

Definition of Execution Hallucinations

Execution hallucinations refer to the phenomenon where LLM-based agents claim to have completed certain sub-stages during the execution phase, but in reality, they have not actually been performed or accomplished. According to the above two sub-stages, we divide execution hallucinations into two types: **Tool Selection Hallucinations (TSHs)**, where non-existent or irrelevant tools are chosen overconfidently; and **Tool Calling Hallucinations (TCHs)**, involving incorrect, omitted, or fabricated parameter fillings.

B. Execution Hallucinations

The execution phase is where the agent translates its deliberated plan p_t into a concrete, executable action a_t. a_t typically involves invoking external tools, either as a single operation or through multiple parallel instances [84]. Broadly, this execution process in Eq.(2) can be further decomposed into two sub-stages [85]:

- **Tool Selection**: Given p_t and a set of candidate tools $\mathcal{T}_{\text{cand}}$, the agent must first select the appropriate tool $T_s \in \mathcal{T}_{\text{cand}}$ as follows[3],

$$T_s = \text{Select}\left(b_t, p_t, \mathcal{T}_{\text{cand}}\right). \tag{12}$$

As there are a large number of tools, $\mathcal{T}_{\text{cand}}$ is typically retrieved from a full tool set $\mathcal{T}$ to narrow the selection scope [86]:

$$\mathcal{T}_{\text{cand}} = \text{Retr}\left(b_t, p_t, \mathcal{T}\right). \tag{13}$$

- **Tool Calling**: Once T_s is selected, the agent then needs to populate T_s with the tool parameters derived from b_t and p_t to form the final executable action a_t:

$$a_t = \text{Call}\left(b_t, p_t, T_s\right). \tag{14}$$

The underlying causes behind these two execution hallucinations are given in the following sub-sections.

1) Tool Documentation Limitation. Execution hallucinations often arise when the agent's internal representation of tool behaviors diverges from the actual functionality of tools, frequently due to limitation in **Tool Documentation** [87]. These hallucinations occur because the agent believes it has correctly selected or invoked a tool, even though its decision is based on misleading documentation. Such deficiencies may include redundant information [88], [89], incomplete or inaccurate descriptions [90], [91], or a lack of standardization [92], [93], all of which impair the agent's ability to properly use tools.

2) Shallow Pattern Understanding. The agent's shallow understanding of **Tool Patterns** may lead to execution hallucinations wherein the agent confidently invokes invalid or outdated tools, mistakenly assuming successful execution. In fact, effective tool usage requires LLM-based agents to develop deep understanding of tool patterns. This requirement primarily stems from two key factors: First, tools often provide multiple functionalities that can be applied across diverse scenarios, rather than being limited to a single task-specific use case [94]; Second, tools frequently exhibit complex collaborative patterns, such as sequential dependencies and the need for nested calls [95]. However, LLM-based agents are typically trained with insufficient exposure to diverse and complex tool-use scenarios [96], [97], [98]. This limitation makes agents prone to hallucinating tool invocations that may appear plausible or confident, but in fact violate expected patterns or omit essential details, especially when dealing with complex and novel tasks [99].

3) Weak Dynamic Adaptability. LLM-based agents are typically trained on relatively static datasets, with their tool-use knowledge embedded in fixed model parameters [97], [98]. However, the execution environment of tools is inherently dynamic and continuously evolving. Tool functionalities may evolve, API interfaces may be modified, and in some cases, tools may be deprecated or replaced [99], [100], [101]. When an agent lacks sufficient adaptability to these **Tool Dynamics**, its tool-use behavior becomes misaligned with the actual environment, thus leading to execution hallucinations wherein the agent confidently invokes outdated or invalid tools while incorrectly assuming successful execution.

[3] T_s may denote multiple tools that are invoked in parallel to accelerate execution.